# Survey Forms (Google Forms Clone)

Build Guide: Project Creation, File Map, Integration and Requirements

Written by Kasinadh Sarma

13 October 2026

## Contents

**1 Overview** **2**

**2 Requirements** **2**

2.1 Software . . . 2

2.2 Packages . . . 2

2.3 Why these libraries . . . 3

2.4 Accounts and configuration . . . 4

**3 How the Layers Connect** **5**

**4 Step-by-Step Build** **5**

**5 What Was Verified** **10**

**6 Known Limits** **11**

**7 Source Code** **12**

7.1 Configuration . . . 12

7.2 Shared API schemas . . . 14

7.3 Server foundations . . . 18

7.4 Repository layer . . . 20

7.5 API routes . . . 32

7.6 Client library . . . 38

7.7 Components . . . 42

7.8 Pages . . . 55

# 1 Overview

This guide explains, in order, how to create the project, which files are created or changed at each step, and how the layers connect. The application lets a user build a survey, share a public link, collect validated responses, and view the submissions.

**Stack.** Next.js (App Router), Tailwind CSS v4, react-hook-form, Zod, `@better-fetch/fetch`, Hono (inside a Next.js route handler), Prisma 7 with PostgreSQL as the primary database, and Cloud Firestore (through the Firebase Admin SDK) as a live mirror of that data and as the store for uploaded files.

**Project layout.** The project uses the root `app/` layout (no `src/` folder). Section 5 says exactly what was checked and what was not.

**Core idea.** The API Zod schemas, kept one file per concern in `shared/schemas/`, are imported by both the browser and the API. The same rules therefore run in the form, in the Hono validator, and before anything reaches a database.

**Decisions in this version.**

- **Hono, not Elysia.** Hono runs inside a Next.js route handler, giving one project and one deployment.
- **Postgres is the source of truth; Firestore mirrors it.** Every write goes to Postgres through Prisma first and is then copied to Firestore under the same ids. If Postgres is unreachable, reads fall back to Firestore. Section 3 explains the design and its limits.
- **Files live in Firestore, not in a storage bucket.** A new Firebase bucket may need a paid plan, so uploaded files are stored as documents in a Firestore collection. This caps a file at 700 KB (Firestore documents are limited to 1 MiB). Moving to a real bucket later changes only `server/repo/files.ts` and the two file routes.

# 2 Requirements

## 2.1 Software

| Requirement | Version | Purpose |
|---|---|---|
| Node.js | 22 or newer (24.18 used here) | Runtime |
| npm | 10 or newer (11.16 used here) | Package manager |
| PostgreSQL | Local, Docker, Neon or Supabase | Primary database used by Prisma |
| Firebase project | Cloud Firestore enabled, Admin SDK service account | Mirror database and file store |
| Git | Any recent version | Version control |

## 2.2 Packages

These are the versions installed in this project on 13 October 2026, read from `node_modules`.

| Package | Version | Role |
|---|---|---|
| `next` | 16.4.0 | Framework (Turbopack, Cache Components on by default) |
| `react`, `react-dom` | 19.3.0 | UI |
| `tailwindcss`, `@tailwindcss/postcss` | 4.3.3 | Styling (v4, CSS-first) |
| `postcss` | 8.5.30 | Used by the Tailwind plugin |
| `zod` | 4.6.7 | Shared schemas and types |
| `react-hook-form` | 7.90.0 | Form state |
| `@hookform/resolvers` | 5.9.1 | Connects Zod to react-hook-form |
| `@better-fetch/fetch` | 1.3.2 | Typed HTTP client |
| `hono` | 4.13.13 | API framework |
| `@hono/zod-validator` | 0.9.1 | Request validation middleware |
| `@prisma/client`, `prisma` | 7.10.0 | Database client, migrations and generation |
| `@prisma/adapter-pg`, `pg` | 7.10.0, 8.23.1 | Postgres driver adapter (required by Prisma 7) |
| `firebase-admin` | 14.5.0 | Firestore access from the server |
| `dotenv` | 18.0.7 | Loads `.env` for `prisma.config.ts` |
| `typescript`, `eslint`, `prettier` | 5.9.3, 9.39.5, 3.9.10 | Types, lint, formatting |

### 2.3 Why these libraries

Each package is here for one job, and the list is kept short on purpose.

| Package | Why it is used |
|---|---|
| `next`, `react`, `react-dom` | One framework for pages, routing and server rendering, and the host for the API route handler. |
| `tailwindcss`, `@tailwindcss/postcss`, `postcss` | Utility-first styling without a separate stylesheet system. v4 is CSS-first, and the PostCSS plugin keeps styling independent of the bundler. |
| `zod` | One schema per concern, shared by the browser and the API, so the form, the validator and the database layer apply the same rules and the TypeScript types are inferred from them. |
| `react-hook-form` | Fast form state for the survey builder and response forms, with few re-renders. |
| `@hookform/resolvers` | Lets react-hook-form use the Zod schemas directly instead of duplicating rules. |
| `@better-fetch/fetch` | A TypeScript-first wrapper over native `fetch`: typed responses and one error shape, without the weight of Axios. |
| `hono` | A small, fast web framework that runs inside a Next.js route handler, so the API ships in the same project and deployment. |
| `@hono/zod-validator` | Validates request bodies, params and queries in Hono with the shared Zod schemas. |
| `@prisma/client`, `prisma` | A type-safe database client generated from the schema, plus migrations. |
| `@prisma/adapter-pg`, `pg` | Prisma 7 requires a driver adapter; this is the node-postgres one for PostgreSQL. |
| `firebase-admin` | Server-side access to Firestore, used as the live mirror and file store. |
| `dotenv` | Loads `.env` for `prisma.config.ts`, which Next.js does not do. |
| `typescript`, `eslint`, `prettier` | Static types, lint rules and consistent formatting. |

**Prisma versions must match.** The npm `latest` tag of `prisma` pointed to an 8.0 release candidate while `@prisma/client` was 7.10.0. The install commands in Step 2 pin the three Prisma packages to 7.10.0. Change all three together when you upgrade.

### 2.4 Accounts and configuration

- A PostgreSQL connection string (`DATABASE_URL`).
- A Firebase project with a Cloud Firestore database (the `(default)` database; the free Spark plan is enough).
- A service account key for the Firebase Admin SDK: project id, client email and private key (Firebase console, Project settings, Service accounts, Generate new private key).
- A random token for the sync endpoints (`SYNC_TOKEN`).

## 3 How the Layers Connect

| Layer | What it does |
|---|---|
| `components/` | Forms, tables and inputs. Validate with the shared Zod schemas before sending. |
| `lib/fetch.ts` | Typed client; every endpoint has input, output, params and query schemas. |
| `server/routes/` | Hono routes. Validate again on the server; return one error shape. |
| `server/repo/forms.ts`, `submissions.ts` | The only entry points the routes use. They combine the two databases. |
| `server/repo/postgres/` | Prisma code: the source of truth. |
| `server/repo/firestore/` | Firestore code: the mirror and the read fallback. |
| `server/repo/files.ts` | Uploaded files, stored in Firestore. |
| `server/repo/sync.ts` | Compares and repairs the two databases. |

**Writes.** `createForm`, `deleteForm` and `createSubmission` run in Postgres first. Only after Postgres has committed does the same row go to Firestore, under the same id. A failure of the Firestore write is logged and does not fail the request, because the response is already correct; the sync endpoint repairs the difference.

**Reads.** Reads come from Postgres. If Postgres throws (for example it is unreachable), the same read runs against Firestore. A "not found" from Postgres is a real answer and is not retried.

**Firestore layout.** A form is one document in `forms/{id}` with its questions embedded and a `responseCount`. Each submission is a document in the subcollection `forms/{id}/submissions/{id}`. Uploaded files are documents in `files/{id}`.

**Counter.** When a submission is mirrored, a transaction writes the document and increments `responseCount` only if the document did not exist, so mirroring the same submission twice counts once.

**Limits of this design.**

- Sync is one-way, from Postgres to Firestore. Edits made directly in Firestore are overwritten by the next sync.
- When Postgres is down, reads work but new forms and submissions fail.
- Syncing is manual (an endpoint you call); there is no scheduler.

## 4 Step-by-Step Build

### Step 1: Create the Next.js project with Tailwind v4

```
npx create-next-app@latest survey-forms --ts --tailwind --eslint --app --no-src-dir --use-npm --
    import-alias "@/*"
cd survey-forms
```

**Files created by the generator:** `package.json`, `tsconfig.json`, `next.config.ts`, `eslint.config.mjs`, `AGENTS.md`, `README.md`, and the folders `app/` and `public/`.

**No `src/` folder.** Every folder in this guide (`app/`, `components/`, `lib/`, `shared/`, `server/`, `prisma/`, `generated/`) sits at the project root. The `@/` alias points to the root through `"paths": { "@/*": ["./*"] }` in `tsconfig.json`, which the starter writes for you.

**Tailwind v4.** `app/globals.css` begins with `@import "tailwindcss";`. There are no `@tailwind` directives and no `tailwind.config.js`. Create `postcss.config.mjs` (listing in Section 7) so the styling does not depend on the starter's Turbopack rule.

**Cache Components.** The starter sets `cacheComponents: true`. Two rules follow, and the code already applies both. A route handler must not export `runtime`. Any page that awaits `params`, and any client component that creates random ids (react-hook-form's field array does), must sit inside a `Suspense` boundary, or `next build` fails.

### Step 2: Install dependencies

Run each command on one line.

```
npm i zod react-hook-form @hookform/resolvers @better-fetch/fetch hono @hono/zod-validator
    firebase-admin pg dotenv @prisma/client@7.10.0 @prisma/adapter-pg@7.10.0
npm i -D prisma@7.10.0 @types/pg prettier
npm i tailwindcss @tailwindcss/postcss postcss
```

### Step 3: Configure environment variables, Firestore and Git

**Files created:** `.env` (never committed) and `.env.example` (committed). Next.js loads `.env`; the Prisma CLI loads it through `dotenv`. Generate the token with `openssl rand -hex 24`.

**Firebase values.** Copy `project_id`, `client_email` and `private_key` from the downloaded service account JSON into `FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL` and `FIREBASE_PRIVATE_KEY`. Keep the private key in double quotes on one line with the line breaks written as `\n`; `server/firestore.ts` turns them back into real line breaks. Then delete the JSON file or keep it outside the project: it grants full admin access.

**Create the database.** In the Firebase console open Firestore and click Create database, accept the `(default)` database and choose a region. Security rules do not matter for this app, because the Admin SDK bypasses them and no browser talks to Firestore.

**File updated:** `.gitignore`. The starter already ignores `.env*`, which also hides the example file. Add:

```
!.env.example
/generated/prisma
*-firebase-adminsdk-*.json
serviceAccount*.json
```

### Step 4: Define the database with Prisma

**Files created:** `prisma.config.ts` and `prisma/schema.prisma`. Prisma 7 reads the database URL from `prisma.config.ts`, writes the client to `generated/prisma`, and needs the `pg` driver adapter at runtime.

| Model | What it represents | Why it is a separate model |
|---|---|---|
| `Form` | One survey (title, description) | Parent record; owns questions and submissions |
| `Question` | One prompt in a form | Needs its own type, options, order and required flag, and answers point to it |
| `Submission` | One respondent's attempt | Groups answers so each response can be counted, paged and deleted as a unit |
| `Answer` | One value for one question | Joins a submission to a question |

```
npx prisma migrate dev --name init
npx prisma generate
```

Application code imports the client as `@/generated/prisma/client`. Do not drop or reset the database schema on a machine that holds real forms; a reset deletes them, and the Firestore mirror is only repopulated from Postgres, not the other way round.

### Step 5: Server foundations

- `server/env.ts`: validates the environment with Zod. Two lazy getters keep the database variables apart from the Firebase ones, so forms keep working when Firebase is not configured.
- `server/db.ts`: one shared Prisma client with the Postgres adapter, kept on `globalThis` so hot reloading does not open new pools.
- `server/firestore.ts`: starts the Firebase Admin app once (reused if it exists) and returns the Firestore database.
- `server/errors.ts`: one error body shape, `{ error: { code, message } }`.
- `server/validate.ts`: a wrapper around `zValidator` that returns the same 400 body with the list of issues.

### Step 6: Write the schemas, one file per concern

- `shared/schemas/question.ts`: question types, the question schema and `isChoiceType`.
- `shared/schemas/form.ts`: create-form payload, form response, dashboard summary and the id parameter.
- `shared/schemas/upload.ts`: allowed file types, the 700 KB limit, the pattern a file id must match, and the upload result.
- `shared/schemas/submission.ts`: the answers payload, the page of submissions and `buildAnswerSchema`.
- `lib/schemas/builder.ts`: the builder screen only; the server never imports it.

`z.email()` and `z.iso.date()` are Zod 4 forms; with an older Zod use `z.string().email()` and `z.string().date()`.

### Step 7: Build the repository layer

- `server/repo/postgres/forms.ts` and `submissions.ts`: all Prisma calls. Forms are parsed with `formSchema` so bad rows are caught on the server. `createSubmission` writes the submission and its answers in one nested create (a single transaction), skips empty answers, stores numbers as numbers, and returns what it stored so the same row can be mirrored.
- `server/repo/firestore/forms.ts` and `submissions.ts`: mirror writes, mirror reads and the counter transaction described in Section 3.
- `server/repo/dual.ts`: `mirrorSafely` (log, do not fail) and `readWithFallback`.
- `server/repo/forms.ts` and `submissions.ts`: the facade the routes import. Postgres first, then mirror; Postgres read first, then fallback.
- `server/repo/files.ts`: save, load and existence-check for uploaded files.
- `server/repo/sync.ts`: `getSyncStatus` and `syncAll`.

### Step 8: Build the Hono API

| Method | Path | Input | Result |
|---|---|---|---|
| GET | `/api/forms` | none | 200, forms with counts |
| POST | `/api/forms` | create-form payload | 201, the form; 400 |
| GET | `/api/forms/:id` | none | 200; 404 |
| DELETE | `/api/forms/:id` | none | 204; 404 |
| POST | `/api/forms/:id/submissions` | answers, checked against that form's questions | 201, `{ id }`; 400; 404 |
| GET | `/api/forms/:id/submissions` | `?page&limit` | 200, one page; file answers are download paths |
| POST | `/api/files` | multipart field `file` | 201, `{ id }`; 400 |
| GET | `/api/files/:id` | none | 200, the file as a download; 404 |
| GET | `/api/sync/status` | bearer token | 200, comparison of both databases; 401 |
| POST | `/api/sync` | bearer token | 200, what was copied; 401 |

**405 for every other method.** Each path ends with an `.all(...)` handler that answers any other method with 405 and an `Allow` header. An unknown path returns a JSON 404, and an unexpected failure returns a generic 500 while the real error is only logged. Error bodies look like `{ error: { code, message, issues? } }`.

**Submissions.** The route builds the answer schema from the form's own questions on every request, validates the body with it, checks that every file answer is the id of a file this app stored, and only then saves. A body that is not JSON becomes `null` and fails validation with a 400.

**Files.** The upload route re-checks type and size on the server, because the browser check can be skipped. Downloads are sent as attachments with `X-Content-Type-Options: nosniff`, so an uploaded file is never run as a page on this site.

**Sync routes.** They can rewrite a whole database and reveal counts, so they answer 401 unless the request carries `Authorization: Bearer <SYNC_TOKEN>`. With no token configured they stay closed.

### Step 9: Mount Hono inside Next.js

`app/api/[[...route]]/route.ts` exports GET, POST, PUT, PATCH, DELETE and OPTIONS, all handled by the Hono app. All methods are exported on purpose: Next.js answers an unexported method with its own empty 405, while Hono answers with the JSON 405 and the `Allow` header. There is no `runtime` export, because Cache Components rejects it.

### Step 10: Create the typed HTTP client and class patterns

`lib/fetch.ts` lists every JSON endpoint with its schemas, so a mistyped URL or wrong body is a type error and every response is validated. `getErrorMessage` reads the server's error message. File upload uses a plain `fetch` with `FormData` because a multipart body cannot be described by a Zod input schema; its response is still parsed with a schema. `lib/useLoad.ts` runs an async loader and reports loading, error or data. `lib/responsive/pattrens/forms.ts` holds every Tailwind class string, one object per component; components import their object as `p` and write `className={p.input}`. The folder keeps the spelling `pattrens` used in your other projects.

### Step 11: Build the frontend

| File | Responsibility |
|---|---|
| `app/page.tsx` | Dashboard: "New form" button and the list of forms. |
| `app/forms/new/page.tsx` | Builder page; the builder sits inside `Suspense`. |
| `app/f/[formId]/page.tsx` | Public survey page; params are read inside `Suspense`. |
| `app/forms/[id]/responses/page.tsx` | Responses page; params are read inside `Suspense`. |
| `app/layout.tsx` | Root layout. `<body>` has `suppressHydrationWarning` because browser extensions such as Grammarly add attributes before React loads. |
| `components/FormBuilder.tsx` | Create a form: react-hook-form with a field array and `builderFormSchema`. |
| `components/OptionsEditor.tsx` | Choices of one question: one input per option, add and remove, and a Yes / No preset. |
| `components/QuestionField.tsx` | One input per question type. |
| `components/FileUploadField.tsx` | Checks the file, uploads it to `/api/files`, stores the returned id. |
| `components/SurveyForm.tsx` | Fills in a survey with a schema built from its questions, then submits. |
| `components/SurveyLoader.tsx` | Loads a form for the public page. |
| `components/FormList.tsx` | Dashboard list with links to the survey and its responses. |
| `components/ResponsesView.tsx` | Table with one column per question and paging; file answers are links. |

### Step 12: Verify

```
npx prettier --check .
npx tsc --noEmit
npx eslint .
npm run build
```

```
npm run dev
```

Add a `.prettierignore` listing `.next`, `.agents`, `.claude`, `generated`, `node_modules` and `package-lock.json` so Prettier skips generated files. Do not run `npm run build` while `npm run dev` is serving the same folder: the build replaces the `.next` directory and can stop the dev server.

Then test by hand: create a form with one question of each type, open its public link, submit an invalid answer (an error appears next to that question), submit a valid one, and open the responses page.

### Step 13: Operate the two databases

```
curl -H "Authorization: Bearer $SYNC_TOKEN" localhost:3000/api/sync/status
curl -X POST -H "Authorization: Bearer $SYNC_TOKEN" localhost:3000/api/sync
```

`status` lists form ids missing from Firestore, ids that exist only in Firestore, and both submission counts; `inSync` is true only when all of these match. Comparing ids matters: counts alone say "in sync" when one form was deleted and another added. `POST /api/sync` copies every form and submission from Postgres (documents are replaced, so repeating it is safe), sets each form's counter to the real number, and removes Firestore forms that no longer exist in Postgres. Run it after any Firestore outage, after restoring Postgres from a backup, or when the server log shows "Firestore mirror failed".

## 5 What Was Verified

**Checked in this project:**

- `prettier -check`, `tsc -noEmit`, `eslint .` and `npm run build` all pass.
- API, against a running dev server with a real Postgres (Docker) and a real Firestore database: creating forms; rejecting a choice question with one option; rejecting invalid answers with one message per field; saving a valid submission and listing it with numbers stored as numbers; dashboard counts; 404 and 405 bodies.
- Files: upload of a text file, download as an attachment with `nosniff`, rejection of a disallowed type and of an 800 KB file, rejection of a malformed and of an unknown file id on submit, acceptance of a real id.
- Sync: new writes appear in both databases with equal counts; a form deleted from Firestore and a stray form added were both reported and repaired; deleting a form removes it from both; with the Postgres container stopped, the form list, a form and its submissions were still served from Firestore, and the databases matched again after Postgres returned.
- Browser: the builder creates a multi-choice form using the Yes / No preset; the public survey page renders the question and options; the hydration warning caused by a browser extension no longer shows.

**Not checked:**

- A full click-through of every question type in the browser (date and number inputs, the file input and the validation messages shown next to fields). They were checked through the API and the shared schemas only.
- The "Firestore mirror failed" path with a real Firestore error. Drift was created by editing Firestore by hand, not by a failing write.

- Deployment, concurrent writers, large data volumes, and a Firebase Storage bucket (not used).

## 6 Known Limits

- **No login.** Anyone with a form link can answer it, and the forms list, responses and file downloads need no sign-in. File ids are random and unguessable, but the responses endpoint returns them.
- **File size.** Files are limited to 700 KB because they are stored inside Firestore documents. Photos and short PDFs fit; larger files need a real bucket.
- **Orphaned files.** Deleting a form does not delete the files its answers pointed to.
- **One-way sync and manual repair**, as described in Section 3.
- **No Delete button.** Forms can only be deleted through the API.

# 7 Source Code

Every listing below is read directly from the repository when this document is built, so it is the exact content of the file. The code follows these conventions: braces on every `if`, a doc comment on each function, descriptive names, Prettier width 80 with single quotes and trailing commas, and no inline Tailwind classes (they live in the patterns file). Comments marked "Why" explain a decision, not the syntax.

## 7.1 Configuration

**File:** `prisma.config.ts`. Prisma 7 configuration: schema path, migrations path and database URL.

```
import 'dotenv/config';
import { defineConfig, env } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
});
```

**File:** `prisma/schema.prisma`. The four models, with a comment on why each exists.

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

datasource db {
  provider = "postgresql"
}

/// Form: one survey created by an author. It holds the title and
/// description, and is the parent of its questions and its submissions.
/// Deleting a form cascades to everything below it.
model Form {
  id          String       @id @default(cuid())
  title       String
  description String?
  published   Boolean      @default(true)
  createdAt   DateTime     @default(now())
  questions   Question[]
  submissions Submission[]
}

/// Question: one prompt inside a form. It is stored in its own table
/// (not as JSON on Form) so questions can be ordered, validated and
/// referenced by answers.
model Question {
  id       String   @id @default(cuid())
  formId   String
  form     Form     @relation(fields: [formId], references: [id], onDelete: Cascade)
  label    String
  /// short_text, long_text, number, email, single_choice, multi_choice,
  /// date, file. Checked by the Zod enum at the API, kept as text in the DB.
  type     String
```

```
  required Boolean  @default(false)
  /// Choices for single_choice and multi_choice; empty for other types
  options  String[]
  /// Position of the question inside the form
  order    Int
  answers  Answer[]

  @@index([formId, order])
}

/// Submission: one respondent's completed attempt at a form. It groups
/// that person's answers together and records when they submitted.
model Submission {
  id        String   @id @default(cuid())
  formId    String
  form      Form     @relation(fields: [formId], references: [id], onDelete: Cascade)
  createdAt DateTime @default(now())
  answers   Answer[]

  @@index([formId, createdAt])
}

/// Answer: the value given to one question inside one submission. It
/// links a Submission to a Question, so every response keeps its
/// question and can be shown in a table or exported.
model Answer {
  id           String     @id @default(cuid())
  submissionId String
  submission   Submission @relation(fields: [submissionId], references: [id], onDelete: Cascade)
  questionId   String
  question     Question   @relation(fields: [questionId], references: [id], onDelete: Cascade)
  /// JSON so one column can hold a string, a number, a string array,
  /// or the id of an uploaded file
  value        Json

  /// A submission can answer each question only once
  @@unique([submissionId, questionId])
}
```

**File:** `postcss.config.mjs`. Tailwind v4 PostCSS plugin.

```
const config = {
  plugins: {
    '@tailwindcss/postcss': {},
  },
};

export default config;
```

**File:** `.env.example`. Environment variable template.

```
DATABASE_URL=postgresql://user:pass@host:5432/surveys
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
# paste the service-account private key with newlines written as \n
FIREBASE_PRIVATE_KEY=
# bearer token for POST /api/sync and GET /api/sync/status
SYNC_TOKEN=
```

**File:** `.prettierrc`. Prettier settings: width 80, single quotes, trailing commas.

```
{
  "printWidth": 80,
```

```
  "singleQuote": true,
  "trailingComma": "all"
}
```

## 7.2 Shared API schemas

**File:** `shared/schemas/question.ts`. Question types, schema and helper.

```
import { z } from 'zod';

// single source of truth: the type dropdown, the Zod enum and the
// TypeScript type are all derived from this one array
export const QUESTION_TYPES = [
  'short_text',
  'long_text',
  'number',
  'email',
  'single_choice',
  'multi_choice',
  'date',
  'file',
] as const;
export const questionTypeSchema = z.enum(QUESTION_TYPES);
export type QuestionType = z.infer<typeof questionTypeSchema>;

/**
 * Tells whether a question type is answered by picking from options.
 * Why: the builder (to show the options box) and the schemas (to require
 * options) both need this rule, so it lives in one place.
 */
export function isChoiceType(questionType: QuestionType): boolean {
  return questionType === 'single_choice' || questionType === 'multi_choice';
}

// what the server accepts and stores; shared by client and server so
// both sides apply exactly the same rules
export const questionSchema = z
  .object({
    id: z.string().optional(),
    label: z.string().min(1, 'Question text is required').max(200),
    type: questionTypeSchema,
    required: z.boolean(),
    options: z.array(z.string().min(1)).max(50),
    order: z.number().int().nonnegative(),
  })
  .refine(
    (question) => {
      // a choice question with fewer than 2 options cannot be answered
      // in a meaningful way, so reject it at the API as well
      if (isChoiceType(question.type)) {
        return question.options.length >= 2;
      }
      return true;
    },
    {
      message: 'Choice questions need at least 2 options',
      path: ['options'],
    },
  );
export type Question = z.infer<typeof questionSchema>;
```

**File:** `shared/schemas/form.ts`. Create-form payload, form response, dashboard summary.

```
import { z } from 'zod';
import { questionSchema } from '@/shared/schemas/question';
export const createFormSchema = z.object({
  title: z.string().min(1, 'Title is required').max(120),
  description: z.string().max(1000).optional(),
  questions: z
    .array(questionSchema)
    .min(1, 'Add at least one question')
    .max(100),
});
// the API response is the create payload plus the fields the
// database generates (id, createdAt)
export const formSchema = createFormSchema.extend({
  id: z.string(),
  createdAt: z.string(),
});
// one row of the dashboard list: no questions, but a response count
export const formSummarySchema = z.object({
  id: z.string(),
  title: z.string(),
  createdAt: z.string(),
  responseCount: z.number().int().nonnegative(),
});
export const formListSchema = z.array(formSummarySchema);
export const formIdParamSchema = z.object({
  id: z.string().min(1),
});
export type CreateFormInput = z.infer<typeof createFormSchema>;
export type Form = z.infer<typeof formSchema>;
export type FormSummary = z.infer<typeof formSummarySchema>;
```

**File:** `shared/schemas/upload.ts`. Upload limits, allowed types, file id pattern.

```
import { z } from 'zod';

// Firestore documents are limited to 1 MiB and the file is stored as
// bytes inside one, so the limit stays well under that
export const MAX_UPLOAD_BYTES = 700 * 1024;

export const ALLOWED_UPLOAD_TYPES = [
  'image/png',
  'image/jpeg',
  'image/webp',
  'application/pdf',
  'text/plain',
] as const;

// ids are created by the upload route (Firestore auto ids), so an answer
// can only point to a file this app issued
export const FILE_ID_PATTERN = /^[A-Za-z0-9]{20}$/;

export const fileMetaSchema = z.object({
  filename: z.string().min(1).max(200),
  contentType: z.enum(ALLOWED_UPLOAD_TYPES),
  size: z.number().int().positive().max(MAX_UPLOAD_BYTES),
});

export const fileIdParamSchema = z.object({
  id: z.string().regex(FILE_ID_PATTERN),
});

export const uploadResultSchema = z.object({
  id: z.string(),
});
```

```
export type FileMeta = z.infer<typeof fileMetaSchema>;
export type UploadResult = z.infer<typeof uploadResultSchema>;
```

**File:** `shared/schemas/submission.ts`. Answers, submissions page, and the runtime answer schema.

```
import { z } from 'zod';

import type { Question } from '@/shared/schemas/question';
import { FILE_ID_PATTERN } from '@/shared/schemas/upload';

// every answer is a string, except multiple choice which is a string
// array; the server converts number answers when it saves them
export const answersInputSchema = z.record(
  z.string(),
  z.union([z.string(), z.array(z.string())]),
);

export const submissionResultSchema = z.object({
  id: z.string(),
});

export type AnswerValues = z.infer<typeof answersInputSchema>;

// what is stored for one answer; a file answer is the file id when saved
// and a download path when returned by the list endpoint
export const storedValueSchema = z.union([
  z.string(),
  z.number(),
  z.array(z.string()),
]);

export const submissionRowSchema = z.object({
  id: z.string(),
  createdAt: z.string(),
  answers: z.record(z.string(), storedValueSchema.nullable()),
});

export const submissionsPageSchema = z.object({
  items: z.array(submissionRowSchema),
  total: z.number().int().nonnegative(),
  page: z.number().int(),
  limit: z.number().int(),
});

export const submissionsQuerySchema = z.object({
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20),
});

export type StoredValue = z.infer<typeof storedValueSchema>;
export type SubmissionRow = z.infer<typeof submissionRowSchema>;
export type SubmissionsPage = z.infer<typeof submissionsPageSchema>;

const REQUIRED_MESSAGE = 'This question is required';

/**
 * Returns the key used for a question's input. Falls back to the
 * question order when the question has no id yet.
 */
export function getFieldName(question: Question): string {
  return question.id ?? String(question.order);
}
```

```
/**
 * Builds the validation schema for a single question's answer.
 */
function buildQuestionSchema(question: Question): z.ZodType {
  const choices = question.options as [string, ...string[]];

  if (question.type === 'multi_choice') {
    const multiSchema = z.array(z.enum(choices));
    return question.required
      ? multiSchema.min(1, REQUIRED_MESSAGE)
      : multiSchema;
  }

  // every single-value answer arrives as a string, so the input type is
  // string; this is what lets .pipe() accept it below
  let baseSchema: z.ZodType<unknown, string>;

  switch (question.type) {
    case 'email':
      baseSchema = z.email('Enter a valid email address');
      break;
    case 'date':
      baseSchema = z.iso.date('Enter a valid date');
      break;
    case 'number':
      // kept as a string so an empty box is not silently turned into 0
      baseSchema = z.string().refine((value) => !Number.isNaN(Number(value)), {
        message: 'Enter a valid number',
      });
      break;
    case 'single_choice':
      baseSchema = z.enum(choices, { error: 'Pick one option' });
      break;
    case 'file':
      // the answer is the id returned by the upload route
      baseSchema = z.string().regex(FILE_ID_PATTERN, 'Upload a file');
      break;
    default:
      baseSchema = z.string().max(5000, 'Answer is too long');
  }

  if (question.required) {
    // check "not empty" first so the user sees "required" instead of
    // a confusing message such as "invalid email"
    return z.string().min(1, REQUIRED_MESSAGE).pipe(baseSchema);
  }

  // empty inputs arrive as '', so an optional field must accept ''
  return z.union([z.literal(''), baseSchema]).optional();
}

/**
 * Builds the schema for a whole survey from its questions. The same
 * function runs in the browser and on the server.
 */
export function buildAnswerSchema(questions: Question[]) {
  const shape: Record<string, z.ZodType> = {};

  for (const question of questions) {
    shape[getFieldName(question)] = buildQuestionSchema(question);
  }

  return z.object(shape);
}
```

### 7.3 Server foundations

**File:** `server/env.ts`. Lazy, validated environment variables.

```
import { z } from 'zod';

const databaseEnvSchema = z.object({
  DATABASE_URL: z.string().min(1),
});

const firebaseEnvSchema = z.object({
  FIREBASE_PROJECT_ID: z.string().min(1),
  FIREBASE_CLIENT_EMAIL: z.string().min(1),
  FIREBASE_PRIVATE_KEY: z.string().min(1),
});

type DatabaseEnv = z.infer<typeof databaseEnvSchema>;
type FirebaseEnv = z.infer<typeof firebaseEnvSchema>;

let cachedDatabaseEnv: DatabaseEnv | undefined;
let cachedEnv: FirebaseEnv | undefined;

/**
 * Reads and validates the database variables on first use. Kept apart
 * from the Firebase ones so forms keep working when file storage is not
 * configured.
 */
export function getDatabaseEnv(): DatabaseEnv {
  if (!cachedDatabaseEnv) {
    cachedDatabaseEnv = databaseEnvSchema.parse(process.env);
  }

  return cachedDatabaseEnv;
}

/**
 * Reads and validates the Firebase variables on first use.
 * Why lazy: importing this file during a build or a test must not fail
 * when the variables are not set; only real use needs them.
 */
export function getFirebaseEnv(): FirebaseEnv {
  if (!cachedEnv) {
    cachedEnv = firebaseEnvSchema.parse(process.env);
  }

  return cachedEnv;
}
```

**File:** `server/db.ts`. Shared Prisma client with the Postgres adapter.

```
import { PrismaPg } from '@prisma/adapter-pg';

import { PrismaClient } from '@/generated/prisma/client';
import { getDatabaseEnv } from '@/server/env';

const globalForPrisma = globalThis as unknown as {
  prisma?: PrismaClient;
};

/**
 * Creates a Prisma client that talks to PostgreSQL through the pg
 * driver adapter (required by Prisma 7).
 */
function createPrismaClient(): PrismaClient {
```

```
  const adapter = new PrismaPg({
    connectionString: getDatabaseEnv().DATABASE_URL,
  });

  return new PrismaClient({ adapter });
}

/**
 * Returns one shared Prisma client.
 * Why a singleton on globalThis: hot reloading in development would
 * otherwise open a new connection pool on every file change.
 */
export function getPrisma(): PrismaClient {
  if (!globalForPrisma.prisma) {
    globalForPrisma.prisma = createPrismaClient();
  }

  return globalForPrisma.prisma;
}
```

**File:** `server/firestore.ts`. Firebase Admin app and Firestore database.

```
import { cert, getApps, initializeApp } from 'firebase-admin/app';
import { getFirestore, type Firestore } from 'firebase-admin/firestore';

import { getFirebaseEnv } from '@/server/env';

/**
 * Returns the shared Firestore database. The admin app is created on
 * first use; getApps() makes this safe when hot reloading re-runs the
 * module, because the app already exists then.
 * Why the replace: a private key stored in .env has its newlines
 * written as the two characters \n.
 */
export function getDb(): Firestore {
  if (getApps().length === 0) {
    const env = getFirebaseEnv();

    initializeApp({
      credential: cert({
        projectId: env.FIREBASE_PROJECT_ID,
        clientEmail: env.FIREBASE_CLIENT_EMAIL,
        privateKey: env.FIREBASE_PRIVATE_KEY.replace(/\\n/g, '\n'),
      }),
    });
  }

  return getFirestore();
}
```

**File:** `server/errors.ts`. The one error body shape.

```
/**
 * Builds the JSON body every error response uses, so the client can
 * always read error.message and error.code in the same place.
 */
export function errorBody(code: string, message: string) {
  return {
    error: {
      code,
      message,
    },
  };
}
```

**File:** `server/validate.ts`. zValidator wrapper with a consistent 400 body.

```
import { zValidator } from '@hono/zod-validator';
import type { ValidationTargets } from 'hono';
import type { ZodType } from 'zod';

/**
 * Wraps zValidator so a failed validation always returns the same
 * 400 body, with the list of issues, instead of the library default.
 */
export function validate<
  Target extends keyof ValidationTargets,
  Schema extends ZodType,
>(target: Target, schema: Schema) {
  return zValidator(target, schema, (result, context) => {
    if (!result.success) {
      return context.json(
        {
          error: {
            code: 'VALIDATION_ERROR',
            message: 'The request is not valid',
            issues: result.error.issues,
          },
        },
        400,
      );
    }
  });
}
```

## 7.4 Repository layer

**File:** `server/repo/postgres/forms.ts`. Form data access in Postgres (source of truth).

```
import { getPrisma } from '@/server/db';
import {
  formSchema,
  type CreateFormInput,
  type Form,
  type FormSummary,
} from '@/shared/schemas/form';

/**
 * Turns a database row into the Form shape. The result is parsed with
 * formSchema so bad data in the database is caught here, not in the UI.
 */
function toForm(row: {
  id: string;
  title: string;
  description: string | null;
  createdAt: Date;
  questions: {
    id: string;
    label: string;
    type: string;
    required: boolean;
    options: string[];
    order: number;
  }[];
}): Form {
```

```
  return formSchema.parse({
    id: row.id,
    title: row.title,
    description: row.description ?? undefined,
    createdAt: row.createdAt.toISOString(),
    questions: row.questions,
  });
}

/**
 * Saves a form together with its questions in one write.
 */
export async function createForm(input: CreateFormInput): Promise<Form> {
  const created = await getPrisma().form.create({
    data: {
      title: input.title,
      description: input.description,
      questions: {
        create: input.questions.map((question) => {
          return {
            label: question.label,
            type: question.type,
            required: question.required,
            options: question.options,
            order: question.order,
          };
        }),
      },
    },
    include: {
      questions: { orderBy: { order: 'asc' } },
    },
  });

  return toForm(created);
}

/**
 * Loads one form with its questions in order, or null when not found.
 */
export async function getForm(id: string): Promise<Form | null> {
  const found = await getPrisma().form.findUnique({
    where: { id },
    include: {
      questions: { orderBy: { order: 'asc' } },
    },
  });

  if (!found) {
    return null;
  }

  return toForm(found);
}

/**
 * Lists all forms, newest first, with the number of responses.
 */
export async function listForms(): Promise<FormSummary[]> {
  const rows = await getPrisma().form.findMany({
    orderBy: { createdAt: 'desc' },
    include: {
      _count: { select: { submissions: true } },
    },
  });
```

```
  return rows.map((row) => {
    return {
      id: row.id,
      title: row.title,
      createdAt: row.createdAt.toISOString(),
      responseCount: row._count.submissions,
    };
  });
}

/**
 * Deletes a form. Questions, submissions and answers are removed by
 * the cascade rules in the schema. Returns false when nothing matched.
 */
export async function deleteForm(id: string): Promise<boolean> {
  const result = await getPrisma().form.deleteMany({
    where: { id },
  });

  return result.count > 0;
}
```

**File:** `server/repo/postgres/submissions.ts`. Submission data access in Postgres.

```
import { getPrisma } from '@/server/db';
import type { Form } from '@/shared/schemas/form';
import {
  getFieldName,
  storedValueSchema,
  type StoredValue,
  type SubmissionsPage,
} from '@/shared/schemas/submission';

type ParsedAnswers = Record<string, unknown>;

export type MirroredSubmission = {
  id: string;
  createdAt: Date;
  answers: Record<string, StoredValue>;
};

/**
 * Tells whether an answer was left empty, so it is not stored at all.
 */
function isEmptyAnswer(rawValue: unknown): boolean {
  if (rawValue === undefined || rawValue === '') {
    return true;
  }

  return Array.isArray(rawValue) && rawValue.length === 0;
}

/**
 * Converts a validated answer to the value that is stored. Number
 * questions arrive as strings and are stored as real numbers.
 */
function toStoredValue(
  questionType: string,
  rawValue: unknown,
): StoredValue | null {
  if (questionType === 'number') {
    return Number(rawValue);
  }
```

```
  const parsed = storedValueSchema.safeParse(rawValue);

  return parsed.success ? parsed.data : null;
}

/**
 * Saves one submission and its answers in a single transaction, so a
 * response is never stored half complete. Returns what was stored.
 */
export async function createSubmission(
  form: Form,
  answers: ParsedAnswers,
): Promise<MirroredSubmission> {
  const rows: { questionId: string; value: StoredValue }[] = [];

  for (const question of form.questions) {
    const rawValue = answers[getFieldName(question)];

    if (!question.id || isEmptyAnswer(rawValue)) {
      continue;
    }

    const value = toStoredValue(question.type, rawValue);

    if (value !== null) {
      rows.push({ questionId: question.id, value });
    }
  }

  const created = await getPrisma().submission.create({
    data: {
      formId: form.id,
      answers: { create: rows },
    },
    select: { id: true, createdAt: true },
  });

  // the stored answers are returned too, so the same row can be
  // mirrored to the second database without reading it back
  const stored: Record<string, StoredValue> = {};

  for (const row of rows) {
    stored[row.questionId] = row.value;
  }

  return { id: created.id, createdAt: created.createdAt, answers: stored };
}

/**
 * Returns one page of submissions, newest first. Each answer is keyed
 * by question id so the UI can place it in the right table column.
 */
export async function listSubmissions(
  formId: string,
  page: number,
  limit: number,
): Promise<SubmissionsPage> {
  const prisma = getPrisma();

  const [total, rows] = await Promise.all([
    prisma.submission.count({ where: { formId } }),
    prisma.submission.findMany({
      where: { formId },
      orderBy: { createdAt: 'desc' },
```

```
      skip: (page - 1) * limit,
      take: limit,
      include: { answers: true },
    }),
  ]);

  const items = rows.map((row) => {
    const answers: Record<string, StoredValue | null> = {};

    for (const answer of row.answers) {
      const parsed = storedValueSchema.safeParse(answer.value);
      answers[answer.questionId] = parsed.success ? parsed.data : null;
    }

    return {
      id: row.id,
      createdAt: row.createdAt.toISOString(),
      answers,
    };
  });

  return { items, total, page, limit };
}
```

**File:** `server/repo/firestore/forms.ts`. Form mirror writes and reads in Firestore.

```
import { Timestamp } from 'firebase-admin/firestore';

import { getDb } from '@/server/firestore';
import { formSchema, type Form, type FormSummary } from '@/shared/schemas/form';

/**
 * Turns a Firestore document into the Form shape. The result is parsed
 * with formSchema, the same check the Postgres side applies.
 */
function toForm(id: string, data: FirebaseFirestore.DocumentData): Form {
  return formSchema.parse({
    id,
    title: data.title,
    description: data.description ?? undefined,
    createdAt: (data.createdAt as Timestamp).toDate().toISOString(),
    questions: data.questions,
  });
}

/**
 * Writes a form into Firestore under the same id it has in Postgres.
 * Questions are embedded because they are always read with the form.
 * set() replaces the whole document, so running this twice is safe.
 */
export async function mirrorForm(
  form: Form,
  responseCount: number,
): Promise<void> {
  await getDb()
    .collection('forms')
    .doc(form.id)
    .set({
      title: form.title,
      // Firestore rejects undefined, so the field is left out when empty
      ...(form.description ? { description: form.description } : {}),
      createdAt: Timestamp.fromDate(new Date(form.createdAt)),
      responseCount,
      questions: form.questions,
```

```
    });
}

/**
 * Removes a form and its submissions from Firestore. Firestore does not
 * cascade, so recursiveDelete takes the subcollection with it.
 */
export async function removeFormMirror(id: string): Promise<void> {
  const db = getDb();

  await db.recursiveDelete(db.collection('forms').doc(id));
}

/**
 * Loads one form from Firestore, or null when not found.
 */
export async function getFormMirror(id: string): Promise<Form | null> {
  const snapshot = await getDb().collection('forms').doc(id).get();
  const data = snapshot.data();

  return data ? toForm(snapshot.id, data) : null;
}

/**
 * Lists the forms in Firestore, newest first, with response counts.
 */
export async function listFormMirrors(): Promise<FormSummary[]> {
  const snapshot = await getDb()
    .collection('forms')
    .orderBy('createdAt', 'desc')
    .get();

  return snapshot.docs.map((doc) => {
    const data = doc.data();

    return {
      id: doc.id,
      title: data.title,
      createdAt: (data.createdAt as Timestamp).toDate().toISOString(),
      responseCount: data.responseCount ?? 0,
    };
  });
}

/**
 * Returns the ids of every form in Firestore.
 */
export async function listMirroredFormIds(): Promise<string[]> {
  const snapshot = await getDb().collection('forms').select().get();

  return snapshot.docs.map((doc) => doc.id);
}
```

**File:** `server/repo/firestore/submissions.ts`. Submission mirror writes and reads in Firestore.

```
import { FieldValue, Timestamp } from 'firebase-admin/firestore';

import { getDb } from '@/server/firestore';
import {
  storedValueSchema,
  type StoredValue,
  type SubmissionsPage,
} from '@/shared/schemas/submission';
```

```
export type SubmissionMirror = {
  id: string;
  createdAt: Date;
  answers: Record<string, StoredValue>;
};

/**
 * Writes one submission into Firestore under the id it has in Postgres,
 * and counts it on the form. A transaction checks whether the document
 * already exists, so mirroring the same submission twice counts once.
 */
export async function mirrorSubmission(
  formId: string,
  submission: SubmissionMirror,
): Promise<void> {
  const db = getDb();
  const formRef = db.collection('forms').doc(formId);
  const submissionRef = formRef.collection('submissions').doc(submission.id);

  await db.runTransaction(async (transaction) => {
    const existing = await transaction.get(submissionRef);

    transaction.set(submissionRef, {
      createdAt: Timestamp.fromDate(submission.createdAt),
      answers: submission.answers,
    });

    if (!existing.exists) {
      transaction.update(formRef, { responseCount: FieldValue.increment(1) });
    }
  });
}

/**
 * Writes one submission document without touching the form's counter.
 * Used by the full sync, which sets the counter itself.
 */
export async function writeSubmission(
  formId: string,
  submission: SubmissionMirror,
): Promise<void> {
  await getDb()
    .collection('forms')
    .doc(formId)
    .collection('submissions')
    .doc(submission.id)
    .set({
      createdAt: Timestamp.fromDate(submission.createdAt),
      answers: submission.answers,
    });
}

/**
 * Returns one page of submissions from Firestore, newest first, in the
 * same shape the Postgres side returns.
 */
export async function listSubmissionMirrors(
  formId: string,
  page: number,
  limit: number,
): Promise<SubmissionsPage> {
  const collection = getDb()
    .collection('forms')
    .doc(formId)
    .collection('submissions');
```

```
  const [countSnapshot, snapshot] = await Promise.all([
    collection.count().get(),
    collection
      .orderBy('createdAt', 'desc')
      .offset((page - 1) * limit)
      .limit(limit)
      .get(),
  ]);

  const items = snapshot.docs.map((doc) => {
    const data = doc.data();
    const answers: Record<string, StoredValue | null> = {};

    for (const [questionId, value] of Object.entries(data.answers ?? {})) {
      const parsed = storedValueSchema.safeParse(value);
      answers[questionId] = parsed.success ? parsed.data : null;
    }

    return {
      id: doc.id,
      createdAt: (data.createdAt as Timestamp).toDate().toISOString(),
      answers,
    };
  });

  return { items, total: countSnapshot.data().count, page, limit };
}
```

**File:** `server/repo/dual.ts`. Safe mirror write and read with fallback.

```
/**
 * Runs the mirror write and keeps a failure from failing the request.
 * Why: Postgres is the source of truth and has already committed, so the
 * response is still correct; the drift is logged and repaired by the
 * sync endpoint.
 */
export async function mirrorSafely(
  description: string,
  write: () => Promise<void>,
): Promise<void> {
  try {
    await write();
  } catch (error) {
    console.error(`Firestore mirror failed: ${description}`, error);
  }
}

/**
 * Reads from the primary database and falls back to the mirror when the
 * primary throws (for example when Postgres is unreachable). A null
 * result is a real answer, "not found", and is not retried.
 */
export async function readWithFallback<T>(
  description: string,
  primary: () => Promise<T>,
  mirror: () => Promise<T>,
): Promise<T> {
  try {
    return await primary();
  } catch (error) {
    console.error(
      `Postgres read failed, using Firestore: ${description}`,
      error,
```

```
    );

    return mirror();
  }
}
```

**File:** `server/repo/forms.ts`. Form facade: Postgres first, then mirror.

```
import { mirrorSafely, readWithFallback } from '@/server/repo/dual';
import {
  getFormMirror,
  listFormMirrors,
  mirrorForm,
  removeFormMirror,
} from '@/server/repo/firestore/forms';
import * as postgres from '@/server/repo/postgres/forms';
import type { CreateFormInput, Form, FormSummary } from '@/shared/schemas/form';

/**
 * Saves a form in Postgres, then mirrors it to Firestore.
 */
export async function createForm(input: CreateFormInput): Promise<Form> {
  const created = await postgres.createForm(input);

  await mirrorSafely(`create form ${created.id}`, () => mirrorForm(created, 0));

  return created;
}

/**
 * Loads one form from Postgres, or from Firestore when Postgres fails.
 */
export async function getForm(id: string): Promise<Form | null> {
  return readWithFallback(
    `get form ${id}`,
    () => postgres.getForm(id),
    () => getFormMirror(id),
  );
}

/**
 * Lists forms from Postgres, or from Firestore when Postgres fails.
 */
export async function listForms(): Promise<FormSummary[]> {
  return readWithFallback('list forms', postgres.listForms, listFormMirrors);
}

/**
 * Deletes a form from Postgres, then from Firestore.
 */
export async function deleteForm(id: string): Promise<boolean> {
  const wasDeleted = await postgres.deleteForm(id);

  if (wasDeleted) {
    await mirrorSafely(`delete form ${id}`, () => removeFormMirror(id));
  }

  return wasDeleted;
}
```

**File:** `server/repo/submissions.ts`. Submission facade: Postgres first, then mirror.

```
import { mirrorSafely, readWithFallback } from '@/server/repo/dual';
import {
```

```
  listSubmissionMirrors,
  mirrorSubmission,
} from '@/server/repo/firestore/submissions';
import * as postgres from '@/server/repo/postgres/submissions';
import type { Form } from '@/shared/schemas/form';
import type { SubmissionsPage } from '@/shared/schemas/submission';

/**
 * Saves a submission in Postgres, then mirrors it to Firestore.
 */
export async function createSubmission(
  form: Form,
  answers: Record<string, unknown>,
): Promise<{ id: string }> {
  const created = await postgres.createSubmission(form, answers);

  await mirrorSafely(`create submission ${created.id}`, () =>
    mirrorSubmission(form.id, created),
  );

  return { id: created.id };
}

/**
 * Lists submissions from Postgres, or from Firestore when Postgres fails.
 */
export async function listSubmissions(
  formId: string,
  page: number,
  limit: number,
): Promise<SubmissionsPage> {
  return readWithFallback(
    `list submissions of ${formId}`,
    () => postgres.listSubmissions(formId, page, limit),
    () => listSubmissionMirrors(formId, page, limit),
  );
}
```

**File:** `server/repo/files.ts`. Uploaded files stored in Firestore.

```
import { Timestamp } from 'firebase-admin/firestore';

import { getDb } from '@/server/firestore';
import type { FileMeta } from '@/shared/schemas/upload';

type StoredFile = FileMeta & { data: Buffer };

/**
 * Saves one uploaded file as a document in the files collection and
 * returns its id. Why a collection and not a bucket: the project uses
 * Firestore only, and the upload limit keeps each file under the 1 MiB
 * document size.
 */
export async function saveFile(meta: FileMeta, data: Buffer): Promise<string> {
  const ref = getDb().collection('files').doc();

  await ref.set({ ...meta, data, createdAt: Timestamp.now() });

  return ref.id;
}

/**
 * Loads one stored file, or null when the id is unknown.
 */
```

```
export async function getFile(id: string): Promise<StoredFile | null> {
  const snapshot = await getDb().collection('files').doc(id).get();
  const data = snapshot.data();

  if (!data) {
    return null;
  }

  return {
    filename: data.filename,
    contentType: data.contentType,
    size: data.size,
    data: Buffer.from(data.data),
  };
}

/**
 * Returns the ids from the list that do not exist, so a submission can
 * never point to a file this app did not store.
 */
export async function findMissingFiles(ids: string[]): Promise<string[]> {
  if (ids.length === 0) {
    return [];
  }

  const db = getDb();
  const snapshots = await db.getAll(
    ...ids.map((id) => db.collection('files').doc(id)),
  );

  return snapshots.filter((snapshot) => !snapshot.exists).map((s) => s.id);
}
```

**File:** `server/repo/sync.ts`. Sync status and full repair.

```
import { getDb } from '@/server/firestore';
import { getPrisma } from '@/server/db';
import {
  listMirroredFormIds,
  mirrorForm,
  removeFormMirror,
} from '@/server/repo/firestore/forms';
import { writeSubmission } from '@/server/repo/firestore/submissions';
import { getForm } from '@/server/repo/postgres/forms';
import {
  storedValueSchema,
  type StoredValue,
} from '@/shared/schemas/submission';

export type SyncReport = {
  formsCopied: number;
  submissionsCopied: number;
  formsRemoved: number;
};

export type SyncStatus = {
  postgres: { forms: number; submissions: number };
  firestore: { forms: number; submissions: number };
  missingInFirestore: string[];
  extraInFirestore: string[];
  inSync: boolean;
};

/**
```

```
 * Compares the two databases: form ids (so a missing form is caught even
 * when the counts happen to match) and submission counts.
 */
export async function getSyncStatus(): Promise<SyncStatus> {
  const prisma = getPrisma();
  const db = getDb();

  const [pgForms, pgSubmissions, fsFormIds, fsSubmissions] = await Promise.all([
    prisma.form.findMany({ select: { id: true } }),
    prisma.submission.count(),
    listMirroredFormIds(),
    db.collectionGroup('submissions').count().get(),
  ]);

  const postgresIds = pgForms.map((form) => form.id);
  const firestoreIds = new Set(fsFormIds);
  const missingInFirestore = postgresIds.filter((id) => !firestoreIds.has(id));
  const extraInFirestore = fsFormIds.filter((id) => !postgresIds.includes(id));

  const postgres = { forms: postgresIds.length, submissions: pgSubmissions };
  const firestore = {
    forms: fsFormIds.length,
    submissions: fsSubmissions.data().count,
  };

  return {
    postgres,
    firestore,
    missingInFirestore,
    extraInFirestore,
    inSync:
      missingInFirestore.length === 0 &&
      extraInFirestore.length === 0 &&
      postgres.submissions === firestore.submissions,
  };
}

/**
 * Makes Firestore match Postgres: copies every form and submission
 * (documents are replaced, so repeating this is safe) and removes forms
 * that no longer exist in Postgres.
 */
export async function syncAll(): Promise<SyncReport> {
  const prisma = getPrisma();
  const report: SyncReport = {
    formsCopied: 0,
    submissionsCopied: 0,
    formsRemoved: 0,
  };

  const forms = await prisma.form.findMany({ select: { id: true } });
  const postgresIds = new Set(forms.map((form) => form.id));

  for (const { id } of forms) {
    const form = await getForm(id);

    if (!form) {
      continue;
    }

    const submissions = await prisma.submission.findMany({
      where: { formId: id },
      include: { answers: true },
    });
```

```
    // the counter is set to the real number here, so the submissions
    // below are written without incrementing it
    await mirrorForm(form, submissions.length);
    report.formsCopied += 1;

    for (const submission of submissions) {
      const answers: Record<string, StoredValue> = {};

      for (const answer of submission.answers) {
        const parsed = storedValueSchema.safeParse(answer.value);

        if (parsed.success) {
          answers[answer.questionId] = parsed.data;
        }
      }

      await writeSubmission(id, {
        id: submission.id,
        createdAt: submission.createdAt,
        answers,
      });
      report.submissionsCopied += 1;
    }
  }

  for (const id of await listMirroredFormIds()) {
    if (!postgresIds.has(id)) {
      await removeFormMirror(id);
      report.formsRemoved += 1;
    }
  }

  return report;
}
```

## 7.5 API routes

**File:** `server/routes/forms.ts`. Form endpoints and 405 handling.

```
import { Hono } from 'hono';

import { errorBody } from '@/server/errors';
import {
  createForm,
  deleteForm,
  getForm,
  listForms,
} from '@/server/repo/forms';
import { validate } from '@/server/validate';
import { createFormSchema, formIdParamSchema } from '@/shared/schemas/form';

export const formsRoutes = new Hono()
  .get('/forms', async (context) => {
    return context.json(await listForms());
  })
  .post('/forms', validate('json', createFormSchema), async (context) => {
    const created = await createForm(context.req.valid('json'));

    return context.json(created, 201);
  })
  .get('/forms/:id', validate('param', formIdParamSchema), async (context) => {
    const { id } = context.req.valid('param');
    const found = await getForm(id);
```

```
    if (!found) {
      return context.json(errorBody('NOT_FOUND', 'Form not found'), 404);
    }

    return context.json(found);
  })
  .delete(
    '/forms/:id',
    validate('param', formIdParamSchema),
    async (context) => {
      const { id } = context.req.valid('param');
      const wasDeleted = await deleteForm(id);

      if (!wasDeleted) {
        return context.json(errorBody('NOT_FOUND', 'Form not found'), 404);
      }

      return context.body(null, 204);
    },
  )
  // every other method on these paths is rejected with 405 and an
  // Allow header, so a wrong verb never reaches the data layer
  .all('/forms', (context) => {
    return context.json(
      errorBody('METHOD_NOT_ALLOWED', 'Use GET or POST on this path'),
      405,
      { Allow: 'GET, POST' },
    );
  })
  .all('/forms/:id', (context) => {
    return context.json(
      errorBody('METHOD_NOT_ALLOWED', 'Use GET or DELETE on this path'),
      405,
      { Allow: 'GET, DELETE' },
    );
  });
```

**File:** `server/routes/submissions.ts`. Submission endpoints and 405 handling.

```
import { Hono } from 'hono';

import { errorBody } from '@/server/errors';
import { findMissingFiles } from '@/server/repo/files';
import { getForm } from '@/server/repo/forms';
import { createSubmission, listSubmissions } from '@/server/repo/submissions';
import { validate } from '@/server/validate';
import { formIdParamSchema } from '@/shared/schemas/form';
import {
  buildAnswerSchema,
  getFieldName,
  submissionsQuerySchema,
  type SubmissionsPage,
} from '@/shared/schemas/submission';

/**
 * Replaces the stored id of every file answer with the path the browser
 * downloads it from.
 */
function withFileUrls(
  page: SubmissionsPage,
  fileFieldNames: string[],
): SubmissionsPage {
  const items = page.items.map((item) => {
```

```
    const answers = { ...item.answers };

    for (const fieldName of fileFieldNames) {
      const fileId = answers[fieldName];

      if (typeof fileId === 'string') {
        answers[fieldName] = `/api/files/${fileId}`;
      }
    }

    return { ...item, answers };
  });

  return { ...page, items };
}

export const submissionsRoutes = new Hono()
  .post(
    '/forms/:id/submissions',
    validate('param', formIdParamSchema),
    async (context) => {
      const { id } = context.req.valid('param');
      const found = await getForm(id);

      if (!found) {
        return context.json(errorBody('NOT_FOUND', 'Form not found'), 404);
      }

      // the schema depends on this form's questions, so it is built per
      // request; a body that is not JSON becomes null and fails below
      const body = await context.req.json().catch(() => null);
      const parsed = buildAnswerSchema(found.questions).safeParse(body);

      if (!parsed.success) {
        return context.json(
          {
            error: {
              code: 'VALIDATION_ERROR',
              message: 'Some answers are not valid',
              issues: parsed.error.issues,
            },
          },
          400,
        );
      }

      // a file answer must be an id this app issued
      const fileIds = found.questions
        .filter((question) => question.type === 'file')
        .map((question) => parsed.data[getFieldName(question)])
        .filter((value): value is string => typeof value === 'string');

      if ((await findMissingFiles(fileIds)).length > 0) {
        return context.json(
          errorBody('VALIDATION_ERROR', 'An uploaded file was not found'),
          400,
        );
      }

      const created = await createSubmission(found, parsed.data);

      return context.json(created, 201);
    },
  )
  .get(
```

```
    '/forms/:id/submissions',
    validate('param', formIdParamSchema),
    validate('query', submissionsQuerySchema),
    async (context) => {
      const { id } = context.req.valid('param');
      const { page, limit } = context.req.valid('query');
      const found = await getForm(id);

      if (!found) {
        return context.json(errorBody('NOT_FOUND', 'Form not found'), 404);
      }

      const fileFieldNames = found.questions
        .filter((question) => question.type === 'file')
        .map((question) => getFieldName(question));

      const result = await listSubmissions(id, page, limit);

      return context.json(withFileUrls(result, fileFieldNames));
    },
  )
  .all('/forms/:id/submissions', (context) => {
    return context.json(
      errorBody('METHOD_NOT_ALLOWED', 'Use GET or POST on this path'),
      405,
      { Allow: 'GET, POST' },
    );
  });
```

**File:** `server/routes/files.ts`. File upload and download.

```
import { Hono } from 'hono';

import { errorBody } from '@/server/errors';
import { getFile, saveFile } from '@/server/repo/files';
import { validate } from '@/server/validate';
import { fileIdParamSchema, fileMetaSchema } from '@/shared/schemas/upload';

export const filesRoutes = new Hono()
  .post('/files', async (context) => {
    const body = await context.req.parseBody().catch(() => null);
    const file = body?.file;

    if (!(file instanceof File)) {
      return context.json(
        errorBody('VALIDATION_ERROR', 'Send the file in a "file" field'),
        400,
      );
    }

    // the browser's type and size are checked again here, because the
    // client-side check can be skipped by calling the API directly
    const meta = fileMetaSchema.safeParse({
      filename: file.name,
      contentType: file.type,
      size: file.size,
    });

    if (!meta.success) {
      return context.json(
        errorBody(
          'VALIDATION_ERROR',
          'Use a PNG, JPEG, WebP, PDF or text file up to 700 KB',
        ),
```

```
        400,
      );
    }

    const bytes = Buffer.from(await file.arrayBuffer());
    const id = await saveFile(meta.data, bytes);

    return context.json({ id }, 201);
  })
  .get('/files/:id', validate('param', fileIdParamSchema), async (context) => {
    const { id } = context.req.valid('param');
    const found = await getFile(id);

    if (!found) {
      return context.json(errorBody('NOT_FOUND', 'File not found'), 404);
    }

    // attachment + nosniff: an uploaded file is downloaded, never run as
    // a page on this site's origin
    return context.body(new Uint8Array(found.data), 200, {
      'Content-Type': found.contentType,
      'Content-Disposition': `attachment; filename="${encodeURIComponent(found.filename)}"`,
      'X-Content-Type-Options': 'nosniff',
    });
  })
  .all('/files', (context) => {
    return context.json(
      errorBody('METHOD_NOT_ALLOWED', 'Use POST on this path'),
      405,
      { Allow: 'POST' },
    );
  });
```

**File:** `server/routes/sync.ts`. Token-protected sync endpoints.

```
import { timingSafeEqual } from 'node:crypto';

import { Hono } from 'hono';

import { errorBody } from '@/server/errors';
import { getSyncStatus, syncAll } from '@/server/repo/sync';

/**
 * Checks the Authorization header against SYNC_TOKEN. The sync routes
 * can rewrite a whole database and reveal counts, so they are closed
 * unless the token is configured and sent.
 */
function isAuthorized(header: string | undefined): boolean {
  const expected = process.env.SYNC_TOKEN;

  if (!expected || !header) {
    return false;
  }

  const sent = Buffer.from(header.replace(/^Bearer /, ''));
  const wanted = Buffer.from(expected);

  return sent.length === wanted.length && timingSafeEqual(sent, wanted);
}

export const syncRoutes = new Hono()
  .use('/sync/*', async (context, next) => {
    if (!isAuthorized(context.req.header('Authorization'))) {
      return context.json(
```

```
        errorBody('UNAUTHORIZED', 'A valid sync token is required'),
        401,
      );
    }

    await next();
  })
  .get('/sync/status', async (context) => {
    return context.json(await getSyncStatus());
  })
  .post('/sync', async (context) => {
    return context.json(await syncAll());
  });
```

**File:** `server/app.ts`. The Hono app: routes, 404 and error handler.

```
import { Hono } from 'hono';

import { errorBody } from '@/server/errors';
import { filesRoutes } from '@/server/routes/files';
import { formsRoutes } from '@/server/routes/forms';
import { submissionsRoutes } from '@/server/routes/submissions';
import { syncRoutes } from '@/server/routes/sync';

export const app = new Hono()
  .basePath('/api')
  .route('/', formsRoutes)
  .route('/', submissionsRoutes)
  .route('/', filesRoutes)
  .route('/', syncRoutes)
  .notFound((context) => {
    return context.json(errorBody('NOT_FOUND', 'Route not found'), 404);
  })
  .onError((error, context) => {
    // the real error is only logged; the client gets a generic message
    console.error(error);

    return context.json(
      errorBody('INTERNAL_ERROR', 'Something went wrong'),
      500,
    );
  });

export type AppType = typeof app;
```

**File:** `app/api/[[...route]]/route.ts`. Mounts Hono inside Next.js.

```
import { app } from '@/server/app';

// every method is exported on purpose: Next.js answers an unexported
// method with its own empty 405, while Hono sends the JSON 405 with an
// Allow header. No `runtime` export: Cache Components rejects it.
const handler = (request: Request) => app.fetch(request);

export {
  handler as GET,
  handler as POST,
  handler as PUT,
  handler as PATCH,
  handler as DELETE,
  handler as OPTIONS,
};
```

### 7.6 Client library

**File:** `lib/fetch.ts`. Typed HTTP client and error message helper.

```
import { createFetch, createSchema } from '@better-fetch/fetch';
import { z } from 'zod';

import {
  createFormSchema,
  formIdParamSchema,
  formListSchema,
  formSchema,
} from '@/shared/schemas/form';
import {
  answersInputSchema,
  submissionResultSchema,
  submissionsPageSchema,
  submissionsQuerySchema,
} from '@/shared/schemas/submission';

// every endpoint is listed with its input and output schema, so a wrong
// URL or body is a type error and every response is validated
const apiSchema = createSchema({
  '@get/forms': {
    output: formListSchema,
  },
  '@post/forms': {
    input: createFormSchema,
    output: formSchema,
  },
  '@get/forms/:id': {
    params: formIdParamSchema,
    output: formSchema,
  },
  '@delete/forms/:id': {
    params: formIdParamSchema,
    output: z.unknown(),
  },
  '@post/forms/:id/submissions': {
    params: formIdParamSchema,
    input: answersInputSchema,
    output: submissionResultSchema,
  },
  '@get/forms/:id/submissions': {
    params: formIdParamSchema,
    query: submissionsQuerySchema.partial(),
    output: submissionsPageSchema,
  },
});

export const $fetch = createFetch({
  baseURL: '/api',
  schema: apiSchema,
  throw: true,
});

/**
 * Reads the message of the server's error body, so the UI can show
 * the same text the API produced. Falls back to a generic message.
 */
export function getErrorMessage(error: unknown): string {
  if (typeof error === 'object' && error !== null && 'error' in error) {
    const body = (error as { error: unknown }).error;

    if (typeof body === 'object' && body !== null && 'error' in body) {
```

```
      const inner = (body as { error: { message?: unknown } }).error;

      if (typeof inner?.message === 'string') {
        return inner.message;
      }
    }
  }

  if (error instanceof Error) {
    return error.message;
  }

  return 'Something went wrong';
}
```

**File:** `lib/useLoad.ts`. Loading hook.

```
'use client';

import { useEffect, useState } from 'react';

import { getErrorMessage } from '@/lib/fetch';

type LoadState<T> =
  | { status: 'loading' }
  | { status: 'error'; message: string }
  | { status: 'done'; data: T };

/**
 * Runs an async loader and reports loading, error or data.
 * Why the result is stored next to the loader that made it: when the
 * loader changes the old result is ignored, so "loading" is derived
 * and no state is set synchronously inside the effect.
 */
export function useLoad<T>(loader: () => Promise<T>): LoadState<T> {
  const [settled, setSettled] = useState<{
    loader: () => Promise<T>;
    state: LoadState<T>;
  } | null>(null);

  useEffect(() => {
    let isCurrent = true;

    loader().then(
      (data) => {
        if (isCurrent) {
          setSettled({ loader, state: { status: 'done', data } });
        }
      },
      (error: unknown) => {
        if (isCurrent) {
          setSettled({
            loader,
            state: { status: 'error', message: getErrorMessage(error) },
          });
        }
      },
    );

    return () => {
      isCurrent = false;
    };
  }, [loader]);
```

```
  if (settled && settled.loader === loader) {
    return settled.state;
  }

  return { status: 'loading' };
}
```

**File:** `lib/schemas/builder.ts`. Builder-only schema and conversion to the API payload.

```
import { z } from 'zod';

import type { CreateFormInput } from '@/shared/schemas/form';
import { isChoiceType, questionTypeSchema } from '@/shared/schemas/question';

// the builder screen only: each option is its own input, stored as
// { value } because react-hook-form field arrays need objects
export const builderQuestionSchema = z
  .object({
    label: z.string().min(1, 'Question text is required').max(200),
    type: questionTypeSchema,
    required: z.boolean(),
    options: z.array(z.object({ value: z.string() })),
  })
  .refine(
    (question) => {
      if (isChoiceType(question.type)) {
        return cleanOptions(question.options).length >= 2;
      }
      return true;
    },
    {
      message: 'Enter at least 2 options',
      path: ['options'],
    },
  );

export const builderFormSchema = z.object({
  title: z.string().min(1, 'Title is required').max(120),
  description: z.string().max(1000),
  questions: z
    .array(builderQuestionSchema)
    .min(1, 'Add at least one question')
    .max(100),
});

export type BuilderFormValues = z.infer<typeof builderFormSchema>;

/**
 * Trims the option inputs and drops the empty ones.
 */
export function cleanOptions(options: { value: string }[]): string[] {
  return options
    .map((option) => option.value.trim())
    .filter((value) => value.length > 0);
}

/**
 * Converts the builder screen's values to the API payload.
 */
export function toCreateFormInput(values: BuilderFormValues): CreateFormInput {
  return {
    title: values.title,
    description: values.description || undefined,
    questions: values.questions.map((question, index) => {
```

```
      return {
        label: question.label,
        type: question.type,
        required: question.required,
        options: isChoiceType(question.type)
          ? cleanOptions(question.options)
          : [],
        order: index,
      };
    }),
  };
}
```

**File:** `lib/responsive/pattrens/forms.ts`. All Tailwind class strings.

```
// every Tailwind class string lives here, one object per component, so
// components stay free of inline classes

const button =
  'rounded-md bg-indigo-600 px-4 py-2 text-sm font-medium text-white hover:bg-indigo-500 disabled:
    opacity-50';
const buttonQuiet =
  'rounded-md border border-zinc-300 px-3 py-1.5 text-sm hover:bg-zinc-100 dark:border-zinc-700
    dark:hover:bg-zinc-800';
const input =
  'w-full rounded-md border border-zinc-300 bg-transparent px-3 py-2 text-sm dark:border-zinc
    -700';
const errorText = 'mt-1 text-sm text-red-600';

export const pagePatterns = {
  main: 'mx-auto w-full max-w-3xl flex-1 px-4 py-10',
  heading: 'text-2xl font-semibold tracking-tight',
  headerRow: 'mb-6 flex items-center justify-between gap-4',
  wideMain: 'mx-auto w-full max-w-6xl flex-1 px-4 py-10',
};

export const formListPatterns = {
  list: 'divide-y divide-zinc-200 rounded-md border border-zinc-200 dark:divide-zinc-800 dark:
    border-zinc-800',
  row: 'flex items-center justify-between gap-4 px-4 py-3',
  title: 'font-medium',
  meta: 'text-sm text-zinc-500',
  actions: 'flex gap-2',
  link: buttonQuiet,
  newButton: button,
  empty: 'text-sm text-zinc-500',
  error: errorText,
};

export const builderPatterns = {
  form: 'space-y-6',
  field: 'space-y-1',
  label: 'block text-sm font-medium',
  input,
  textarea: `${input} min-h-20`,
  card: 'space-y-3 rounded-md border border-zinc-200 p-4 dark:border-zinc-800',
  cardRow: 'flex flex-wrap items-center gap-3',
  select: `${input} sm:w-56`,
  checkboxLabel: 'flex items-center gap-2 text-sm',
  addButton: buttonQuiet,
  removeButton: 'text-sm text-red-600 hover:underline',
  submit: button,
  error: errorText,
  hint: 'text-xs text-zinc-500',
```

```
  success: 'rounded-md border border-green-300 p-4 text-sm',
};

export const optionsEditorPatterns = {
  wrapper: 'space-y-2',
  label: 'block text-sm font-medium',
  row: 'flex items-center gap-2',
  input,
  removeButton: 'text-sm text-red-600 hover:underline',
  actions: 'flex flex-wrap gap-2',
  actionButton: buttonQuiet,
  error: errorText,
};

export const questionFieldPatterns = {
  wrapper: 'space-y-1',
  label: 'block text-sm font-medium',
  requiredMark: 'text-red-600',
  input,
  textarea: `${input} min-h-24`,
  optionRow: 'flex items-center gap-2 text-sm',
  error: errorText,
};

export const fileUploadPatterns = {
  wrapper: 'space-y-1',
  input: 'block w-full text-sm',
  status: 'text-sm text-zinc-500',
  error: errorText,
};

export const surveyPatterns = {
  form: 'space-y-6',
  description: 'text-sm text-zinc-500',
  submit: button,
  error: errorText,
  done: 'rounded-md border border-green-300 p-4 text-sm',
};

export const responsesPatterns = {
  tableWrap:
    'overflow-x-auto rounded-md border border-zinc-200 dark:border-zinc-800',
  table: 'w-full text-left text-sm',
  headCell:
    'whitespace-nowrap bg-zinc-100 px-3 py-2 font-medium dark:bg-zinc-900',
  cell: 'border-t border-zinc-200 px-3 py-2 align-top dark:border-zinc-800',
  link: 'text-indigo-600 underline',
  pager: 'mt-4 flex items-center justify-between text-sm',
  pagerButton: buttonQuiet,
  empty: 'text-sm text-zinc-500',
  error: errorText,
};
```

## 7.7 Components

**File:** `components/FormBuilder.tsx`. Create-form screen.

```
'use client';

import { useRouter } from 'next/navigation';
import { useState } from 'react';
import { zodResolver } from '@hookform/resolvers/zod';
import { useFieldArray, useForm, useWatch } from 'react-hook-form';
```

```
import { OptionsEditor } from '@/components/OptionsEditor';
import { $fetch, getErrorMessage } from '@/lib/fetch';
import { builderPatterns as p } from '@/lib/responsive/pattrens/forms';
import {
  builderFormSchema,
  toCreateFormInput,
  type BuilderFormValues,
} from '@/lib/schemas/builder';
import { isChoiceType, QUESTION_TYPES } from '@/shared/schemas/question';

const EMPTY_QUESTION: BuilderFormValues['questions'][number] = {
  label: '',
  type: 'short_text',
  required: false,
  options: [{ value: '' }, { value: '' }],
};

/**
 * Reads the message of the options field array. react-hook-form puts an
 * array-level error under `root` or directly on the array.
 */
function getOptionsError(errors: unknown): string | undefined {
  const found = errors as
    { root?: { message?: string }; message?: string } | undefined;

  return found?.root?.message ?? found?.message;
}

/**
 * Create a form: title, description and a list of questions.
 */
export function FormBuilder() {
  const router = useRouter();
  const [serverError, setServerError] = useState('');

  const { register, control, handleSubmit, formState } =
    useForm<BuilderFormValues>({
      resolver: zodResolver(builderFormSchema),
      defaultValues: {
        title: '',
        description: '',
        questions: [EMPTY_QUESTION],
      },
    });

  const { fields, append, remove } = useFieldArray({
    control,
    name: 'questions',
  });
  const watchedQuestions = useWatch({ control, name: 'questions' });
  const { errors } = formState;

  async function onSubmit(values: BuilderFormValues) {
    setServerError('');

    try {
      const created = await $fetch('@post/forms', {
        body: toCreateFormInput(values),
      });
      router.push(`/forms/${created.id}/responses`);
    } catch (error) {
      setServerError(getErrorMessage(error));
    }
  }
```

```
return (
  <form className={p.form} onSubmit={handleSubmit(onSubmit)} noValidate>
    <div className={p.field}>
      <label htmlFor="title" className={p.label}>
        Title
      </label>
      <input id="title" className={p.input} {...register('title')} />
      {errors.title ? (
        <p className={p.error}>{errors.title.message}</p>
      ) : null}
    </div>

    <div className={p.field}>
      <label htmlFor="description" className={p.label}>
        Description
      </label>
      <textarea
        id="description"
        className={p.textarea}
        {...register('description')}
      />
      {errors.description ? (
        <p className={p.error}>{errors.description.message}</p>
      ) : null}
    </div>

    {fields.map((field, index) => {
      const questionType = watchedQuestions?.[index]?.type ?? 'short_text';
      const questionErrors = errors.questions?.[index];

      return (
        <div key={field.id} className={p.card}>
          <div className={p.field}>
            <label htmlFor={`label-${index}`} className={p.label}>
              Question {index + 1}
            </label>
            <input
              id={`label-${index}`}
              className={p.input}
              {...register(`questions.${index}.label`)}
            />
            {questionErrors?.label ? (
              <p className={p.error}>{questionErrors.label.message}</p>
            ) : null}
          </div>

          <div className={p.cardRow}>
            <select
              aria-label="Question type"
              className={p.select}
              {...register(`questions.${index}.type`)}
            >
              {QUESTION_TYPES.map((type) => (
                <option key={type} value={type}>
                  {type.replace('_', ' ')}
                </option>
              ))}
            </select>
            <label className={p.checkboxLabel}>
              <input
                type="checkbox"
                {...register(`questions.${index}.required`)}
              />
              Required
```

```
              </label>
              {fields.length > 1 ? (
                <button
                  type="button"
                  className={p.removeButton}
                  onClick={() => remove(index)}
                >
                  Remove
                </button>
              ) : null}
            </div>

            {isChoiceType(questionType) ? (
              <OptionsEditor
                control={control}
                questionIndex={index}
                error={getOptionsError(questionErrors?.options)}
              />
            ) : null}
          </div>
        );
      })}

      {errors.questions?.root ? (
        <p className={p.error}>{errors.questions.root.message}</p>
      ) : null}
      {errors.questions?.message ? (
        <p className={p.error}>{errors.questions.message}</p>
      ) : null}

      <button
        type="button"
        className={p.addButton}
        onClick={() => append(EMPTY_QUESTION)}
      >
        Add question
      </button>

      {serverError ? <p className={p.error}>{serverError}</p> : null}

      <div>
        <button
          type="submit"
          className={p.submit}
          disabled={formState.isSubmitting}
        >
          {formState.isSubmitting ? 'Saving...' : 'Create form'}
        </button>
      </div>
    </form>
  );
}
```

**File:** `components/OptionsEditor.tsx`. Options editor for choice questions.

```
'use client';

import { useFieldArray, type Control } from 'react-hook-form';

import { optionsEditorPatterns as p } from '@/lib/responsive/pattrens/forms';
import type { BuilderFormValues } from '@/lib/schemas/builder';

type OptionsEditorProps = {
  control: Control<BuilderFormValues>;
```

```
  questionIndex: number;
  error?: string;
};

/**
 * Edits the choices of one question: one input per option, with add and
 * remove buttons and a Yes / No preset for the most common case.
 */
export function OptionsEditor({
  control,
  questionIndex,
  error,
}: OptionsEditorProps) {
  const { fields, append, remove, replace } = useFieldArray({
    control,
    name: `questions.${questionIndex}.options`,
  });

  return (
    <div className={p.wrapper}>
      <span className={p.label}>Options</span>
      {fields.map((field, optionIndex) => (
        <div key={field.id} className={p.row}>
          <input
            aria-label={`Option ${optionIndex + 1}`}
            placeholder={`Option ${optionIndex + 1}`}
            className={p.input}
            {...control.register(
              `questions.${questionIndex}.options.${optionIndex}.value`,
            )}
          />
          {fields.length > 2 ? (
            <button
              type="button"
              className={p.removeButton}
              onClick={() => remove(optionIndex)}
            >
              Remove
            </button>
          ) : null}
        </div>
      ))}
      <div className={p.actions}>
        <button
          type="button"
          className={p.actionButton}
          onClick={() => append({ value: '' })}
        >
          Add option
        </button>
        <button
          type="button"
          className={p.actionButton}
          onClick={() => replace([{ value: 'Yes' }, { value: 'No' }])}
        >
          Yes / No
        </button>
      </div>
      {error ? <p className={p.error}>{error}</p> : null}
    </div>
  );
}
```

**File:** `components/QuestionField.tsx`. One input per question type.

```
'use client';

import { Controller, type Control, type FieldValues } from 'react-hook-form';

import { FileUploadField } from '@/components/FileUploadField';
import { questionFieldPatterns as p } from '@/lib/responsive/pattrens/forms';
import type { Question } from '@/shared/schemas/question';
import { getFieldName } from '@/shared/schemas/submission';

type QuestionFieldProps = {
  question: Question;
  control: Control<FieldValues>;
};

/**
 * Renders the right input for one question type and shows its error.
 */
export function QuestionField({ question, control }: QuestionFieldProps) {
  const name = getFieldName(question);
  const inputId = `question-${name}`;

  return (
    <Controller
      control={control}
      name={name}
      render={({ field, fieldState }) => {
        let input;

        switch (question.type) {
          case 'long_text':
            input = (
              <textarea
                id={inputId}
                className={p.textarea}
                {...field}
                value={field.value ?? ''}
              />
            );
            break;
          case 'single_choice':
            input = (
              <div role="radiogroup" aria-labelledby={`${inputId}-label`}>
                {question.options.map((option) => (
                  <label key={option} className={p.optionRow}>
                    <input
                      type="radio"
                      name={name}
                      value={option}
                      checked={field.value === option}
                      onChange={() => field.onChange(option)}
                    />
                    {option}
                  </label>
                ))}
              </div>
            );
            break;
          case 'multi_choice': {
            const selected: string[] = field.value ?? [];

            input = (
              <div>
                {question.options.map((option) => (
                  <label key={option} className={p.optionRow}>
```

```
                <input
                  type="checkbox"
                  checked={selected.includes(option)}
                  onChange={(event) =>
                    field.onChange(
                      event.target.checked
                        ? [...selected, option]
                        : selected.filter((item) => item !== option),
                    )
                  }
                />
                {option}
              </label>
            ))}
          </div>
        );
        break;
      }
      case 'file':
        input = (
          <FileUploadField
            inputId={inputId}
            onUploaded={(key) => field.onChange(key)}
          />
        );
        break;
      default: {
        const htmlTypes = {
          number: 'number',
          email: 'email',
          date: 'date',
        } as const;
        const htmlType =
          htmlTypes[question.type as keyof typeof htmlTypes] ?? 'text';

        input = (
          <input
            id={inputId}
            type={htmlType}
            step={question.type === 'number' ? 'any' : undefined}
            className={p.input}
            {...field}
            value={field.value ?? ''}
          />
        );
      }
    }

    return (
      <div className={p.wrapper}>
        <label
          id={`${inputId}-label`}
          htmlFor={inputId}
          className={p.label}
        >
          {question.label}
          {question.required ? (
            <span className={p.requiredMark}> *</span>
          ) : null}
        </label>
        {input}
        {fieldState.error ? (
          <p className={p.error}>{fieldState.error.message}</p>
        ) : null}
      </div>
```

```
        );
      }}
    />
  );
}
```

**File:** `components/FileUploadField.tsx`. File picker and upload.

```
'use client';

import { useState, type ChangeEvent } from 'react';

import { getErrorMessage } from '@/lib/fetch';
import { fileUploadPatterns as p } from '@/lib/responsive/pattrens/forms';
import { fileMetaSchema, uploadResultSchema } from '@/shared/schemas/upload';

type FileUploadFieldProps = {
  inputId: string;
  onUploaded: (fileId: string) => void;
};

/**
 * Checks the chosen file, uploads it to the API and reports its id.
 * The server checks the type and size again; this check only gives the
 * user a quick answer.
 */
export function FileUploadField({ inputId, onUploaded }: FileUploadFieldProps) {
  const [status, setStatus] = useState('');
  const [problem, setProblem] = useState('');

  async function handleChange(event: ChangeEvent<HTMLInputElement>) {
    const file = event.target.files?.[0];

    if (!file) {
      return;
    }

    setProblem('');
    setStatus('');

    const meta = fileMetaSchema.safeParse({
      filename: file.name,
      contentType: file.type,
      size: file.size,
    });

    if (!meta.success) {
      setProblem('Use a PNG, JPEG, WebP, PDF or text file up to 700 KB');
      return;
    }

    try {
      setStatus('Uploading...');

      const body = new FormData();
      body.append('file', file);

      const response = await fetch('/api/files', { method: 'POST', body });
      const json: unknown = await response.json();

      if (!response.ok) {
        throw new Error(getErrorMessage({ error: { error: json } }));
      }
```

```
      const result = uploadResultSchema.parse(json);

      setStatus(`Uploaded ${file.name}`);
      onUploaded(result.id);
    } catch (error) {
      setStatus('');
      setProblem(error instanceof Error ? error.message : 'Upload failed');
    }
  }

  return (
    <div className={p.wrapper}>
      <input
        id={inputId}
        type="file"
        className={p.input}
        onChange={handleChange}
      />
      {status ? <p className={p.status}>{status}</p> : null}
      {problem ? <p className={p.error}>{problem}</p> : null}
    </div>
  );
}
```

**File:** `components/SurveyForm.tsx`. Public survey form.

```
'use client';

import { useMemo, useState } from 'react';
import { zodResolver } from '@hookform/resolvers/zod';
import { useForm, type FieldValues, type Resolver } from 'react-hook-form';

import { QuestionField } from '@/components/QuestionField';
import { $fetch, getErrorMessage } from '@/lib/fetch';
import { surveyPatterns as p } from '@/lib/responsive/pattrens/forms';
import type { Form } from '@/shared/schemas/form';
import { buildAnswerSchema, getFieldName } from '@/shared/schemas/submission';

type SurveyFormProps = {
  form: Form;
};

/**
 * Fills in a survey. The validation schema is built from the form's
 * own questions, the same function the server runs on submit.
 */
export function SurveyForm({ form }: SurveyFormProps) {
  const [isDone, setIsDone] = useState(false);
  const [serverError, setServerError] = useState('');

  const schema = useMemo(
    () => buildAnswerSchema(form.questions),
    [form.questions],
  );

  const defaultValues = useMemo(() => {
    const values: FieldValues = {};

    for (const question of form.questions) {
      values[getFieldName(question)] =
        question.type === 'multi_choice' ? [] : '';
    }

    return values;
```

```
  }, [form.questions]);

  const { control, handleSubmit, formState } = useForm<FieldValues>({
    resolver: zodResolver(schema as never) as Resolver<FieldValues>,
    defaultValues,
  });

  async function onSubmit(values: FieldValues) {
    setServerError('');

    try {
      await $fetch('@post/forms/:id/submissions', {
        params: { id: form.id },
        body: values,
      });
      setIsDone(true);
    } catch (error) {
      setServerError(getErrorMessage(error));
    }
  }

  if (isDone) {
    return <p className={p.done}>Thank you, your response was recorded.</p>;
  }

  return (
    <form className={p.form} onSubmit={handleSubmit(onSubmit)} noValidate>
      {form.description ? (
        <p className={p.description}>{form.description}</p>
      ) : null}
      {form.questions.map((question) => (
        <QuestionField
          key={getFieldName(question)}
          question={question}
          control={control}
        />
      ))}
      {serverError ? <p className={p.error}>{serverError}</p> : null}
      <button
        type="submit"
        className={p.submit}
        disabled={formState.isSubmitting}
      >
        {formState.isSubmitting ? 'Sending...' : 'Submit'}
      </button>
    </form>
  );
}
```

**File:** `components/SurveyLoader.tsx`. Loads a form for the public page.

```
'use client';

import { useCallback } from 'react';

import { SurveyForm } from '@/components/SurveyForm';
import { $fetch } from '@/lib/fetch';
import { pagePatterns as p } from '@/lib/responsive/pattrens/forms';
import { useLoad } from '@/lib/useLoad';

type SurveyLoaderProps = {
  formId: string;
};
```

```
/**
 * Loads a form for the public page and shows the survey.
 */
export function SurveyLoader({ formId }: SurveyLoaderProps) {
  const loader = useCallback(
    () => $fetch('@get/forms/:id', { params: { id: formId } }),
    [formId],
  );
  const state = useLoad(loader);

  if (state.status === 'loading') {
    return <p>Loading...</p>;
  }

  if (state.status === 'error') {
    return <p role="alert">{state.message}</p>;
  }

  return (
    <>
      <h1 className={p.heading}>{state.data.title}</h1>
      <SurveyForm form={state.data} />
    </>
  );
}
```

**File:** `components/FormList.tsx`. Dashboard list.

```
'use client';

import Link from 'next/link';
import { useCallback } from 'react';

import { $fetch } from '@/lib/fetch';
import { formListPatterns as p } from '@/lib/responsive/pattrens/forms';
import { useLoad } from '@/lib/useLoad';

/**
 * Dashboard list: each form links to its survey and its responses.
 */
export function FormList() {
  const loader = useCallback(() => $fetch('@get/forms'), []);
  const state = useLoad(loader);

  if (state.status === 'loading') {
    return <p className={p.empty}>Loading...</p>;
  }

  if (state.status === 'error') {
    return <p className={p.error}>{state.message}</p>;
  }

  if (state.data.length === 0) {
    return <p className={p.empty}>No forms yet. Create your first one.</p>;
  }

  return (
    <ul className={p.list}>
      {state.data.map((form) => (
        <li key={form.id} className={p.row}>
          <div>
            <p className={p.title}>{form.title}</p>
            <p className={p.meta}>
              {form.responseCount} responses ·{' '}
```

```
              {new Date(form.createdAt).toLocaleDateString()}
            </p>
          </div>
          <div className={p.actions}>
            <Link className={p.link} href={`/f/${form.id}`}>
              Open survey
            </Link>
            <Link className={p.link} href={`/forms/${form.id}/responses`}>
              Responses
            </Link>
          </div>
        </li>
      ))}
    </ul>
  );
}
```

**File:** `components/ResponsesView.tsx`. Responses table with paging.

```
'use client';

import { useCallback, useState } from 'react';

import { $fetch } from '@/lib/fetch';
import { responsesPatterns as p } from '@/lib/responsive/pattrens/forms';
import { useLoad } from '@/lib/useLoad';
import { getFieldName, type StoredValue } from '@/shared/schemas/submission';

const PAGE_SIZE = 20;

type ResponsesViewProps = {
  formId: string;
};

/**
 * Renders one answer: file answers are links, lists are joined.
 */
function renderAnswer(value: StoredValue | null | undefined, isFile: boolean) {
  if (value === null || value === undefined) {
    return '';
  }

  if (isFile && typeof value === 'string') {
    return (
      <a className={p.link} href={value} target="_blank" rel="noreferrer">
        View file
      </a>
    );
  }

  return Array.isArray(value) ? value.join(', ') : String(value);
}

/**
 * Table with one column per question and paging.
 */
export function ResponsesView({ formId }: ResponsesViewProps) {
  const [page, setPage] = useState(1);

  const loader = useCallback(async () => {
    const [form, submissions] = await Promise.all([
      $fetch('@get/forms/:id', { params: { id: formId } }),
      $fetch('@get/forms/:id/submissions', {
        params: { id: formId },
```

```
      query: { page, limit: PAGE_SIZE },
    }),
  ]);

  return { form, submissions };
}, [formId, page]);
const state = useLoad(loader);

if (state.status === 'loading') {
  return <p>Loading...</p>;
}

if (state.status === 'error') {
  return <p className={p.error}>{state.message}</p>;
}

const { form, submissions } = state.data;
const lastPage = Math.max(1, Math.ceil(submissions.total / PAGE_SIZE));

if (submissions.total === 0) {
  return <p className={p.empty}>No responses yet.</p>;
}

return (
  <>
    <div className={p.tableWrap}>
      <table className={p.table}>
        <thead>
          <tr>
            <th className={p.headCell}>Submitted</th>
            {form.questions.map((question) => (
              <th key={getFieldName(question)} className={p.headCell}>
                {question.label}
              </th>
            ))}
          </tr>
        </thead>
        <tbody>
          {submissions.items.map((row) => (
            <tr key={row.id}>
              <td className={p.cell}>
                {new Date(row.createdAt).toLocaleString()}
              </td>
              {form.questions.map((question) => (
                <td key={getFieldName(question)} className={p.cell}>
                  {renderAnswer(
                    row.answers[getFieldName(question)],
                    question.type === 'file',
                  )}
                </td>
              ))}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
    <div className={p.pager}>
      <button
        type="button"
        className={p.pagerButton}
        disabled={page <= 1}
        onClick={() => setPage(page - 1)}
      >
        Previous
      </button>
```

```
        <span>
          Page {page} of {lastPage} · {submissions.total} responses
        </span>
        <button
          type="button"
          className={p.pagerButton}
          disabled={page >= lastPage}
          onClick={() => setPage(page + 1)}
        >
          Next
        </button>
      </div>
    </>
  );
}
```

## 7.8 Pages

**File:** `app/layout.tsx`. Root layout.

```
import type { Metadata } from 'next';
import { Geist, Geist_Mono } from 'next/font/google';
import './globals.css';

const geistSans = Geist({
  variable: '--font-geist-sans',
  subsets: ['latin'],
});

const geistMono = Geist_Mono({
  variable: '--font-geist-mono',
  subsets: ['latin'],
});

export const metadata: Metadata = {
  title: 'Survey Forms',
  description: 'Build surveys, share a link, collect responses',
};

export default function RootLayout({ children }: LayoutProps<'/'>) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col" suppressHydrationWarning>
        {children}
      </body>
    </html>
  );
}
```

**File:** `app/page.tsx`. Dashboard.

```
import Link from 'next/link';
import { Suspense } from 'react';

import { FormList } from '@/components/FormList';
import {
  formListPatterns,
  pagePatterns as p,
} from '@/lib/responsive/pattrens/forms';
```

```
export default function Home() {
  return (
    <main className={p.main}>
      <div className={p.headerRow}>
        <h1 className={p.heading}>Your forms</h1>
        <Link href="/forms/new" className={formListPatterns.newButton}>
          New form
        </Link>
      </div>
      <Suspense fallback={<p>Loading...</p>}>
        <FormList />
      </Suspense>
    </main>
  );
}
```

**File:** `app/forms/new/page.tsx`. Builder page.

```
import { Suspense } from 'react';

import { FormBuilder } from '@/components/FormBuilder';
import { pagePatterns as p } from '@/lib/responsive/pattrens/forms';

export default function NewFormPage() {
  return (
    <main className={p.main}>
      <h1 className={p.heading}>New form</h1>
      {/* the field array creates random ids, so it sits in Suspense */}
      <Suspense fallback={<p>Loading...</p>}>
        <FormBuilder />
      </Suspense>
    </main>
  );
}
```

**File:** `app/f/[formId]/page.tsx`. Public survey page.

```
import { Suspense } from 'react';

import { SurveyLoader } from '@/components/SurveyLoader';
import { pagePatterns as p } from '@/lib/responsive/pattrens/forms';

/**
 * Reads params inside Suspense, as Cache Components requires.
 */
async function SurveyContent({
  params,
}: {
  params: Promise<{ formId: string }>;
}) {
  const { formId } = await params;

  return <SurveyLoader formId={formId} />;
}

export default function SurveyPage(props: PageProps<'/f/[formId]'>) {
  return (
    <main className={p.main}>
      <Suspense fallback={<p>Loading...</p>}>
        <SurveyContent params={props.params} />
      </Suspense>
    </main>
  );
```

```
}
```

**File:** `app/forms/[id]/responses/page.tsx`. Responses page.

```
import { Suspense } from 'react';

import { ResponsesView } from '@/components/ResponsesView';
import { pagePatterns as p } from '@/lib/responsive/pattrens/forms';

/**
 * Reads params inside Suspense, as Cache Components requires.
 */
async function ResponsesContent({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;

  return <ResponsesView formId={id} />;
}

export default function ResponsesPage(
  props: PageProps<'/forms/[id]/responses'>,
) {
  return (
    <main className={p.wideMain}>
      <div className={p.headerRow}>
        <h1 className={p.heading}>Responses</h1>
      </div>
      <Suspense fallback={<p>Loading...</p>}>
        <ResponsesContent params={props.params} />
      </Suspense>
    </main>
  );
}
```